

Video 2: Building the Skill

How to think iteratively with AI

Learning Objectives

By the end of this session, you should be able to:

- Explain why prompting is a structured skill and the risks of monolithic prompts.
- Describe the 4-stage staged prompting workflow (plan → requirements → tasks → implementation).
- Outline the iterative development loop (build, test, inspect, refine).
- Identify the four prompt templates used for planning, implementation, and debugging.
- Recognize how structured prompting improves control, traceability, and reliability of AI-generated code.

Agenda

In this session, we'll discuss:

- Prompting as a Structured Skill
- 4-Stage Staged Prompting Workflow
- Iterative Development Loop
- Prompt Templates Library
- Live Walkthrough: Tax Calculator

Practical Prompt Engineering for Code

Prompting is not about clever phrasing, it is about structure and control

Most Common Mistake: The Monolithic Prompt

Most beginners ask for everything at once. This is the fastest way to get bloated, unreliable, hard-to-debug code.

What Beginners Do

- "Build me a full tax calculator in Python with multiple tax brackets, a GUI, CSV export, previous calculation history, and an email summary feature."

What Goes Wrong

- AI tries to do everything at once—the result is bloated
- Logic is tangled, hard to trace, hard to debug
- Incomplete sections filled with stubs or placeholder code
- One prompt, one review, zero control

Large, monolithic prompts produce large, unreliable code. Smaller, focused prompts consistently yield better results.

4-Stage Staged Prompting Workflow

1

Ask for a Plan

Before any code, get a high-level architecture. What components exist? What should be built first?



2

Refine Into Requirements

Convert the plan into concrete specifications. Define inputs, outputs, edge cases, and constraints.



3

Break Into Tasks

Split each component into individual, small, implementable tasks. One task = one prompt.



4

Implement One Task

Build one task at a time. Test it. Verify it. Only then move to the next task.

This staged approach gives you a checkpoint to review and verify at every step—before errors compound.

Each stage is also a decision point. If the plan is wrong, fix it before spending time on implementation.

Staged Prompting in Action: Planning the Tax Calculator

Stage 1 Prompt: "I want to build a Python tax calculator. Before writing any code, give me a high-level plan—what components do I need and what should I build first?"

AI Returns a Plan

- Component 1: Tax bracket data structure
- Component 2: `calculate_tax(income)` function
- Component 3: Input validation
- Component 4: Output formatting and display
- Component 5: (Optional) CSV export

Suggested order: 1 → 3 → 2 → 4 → 5

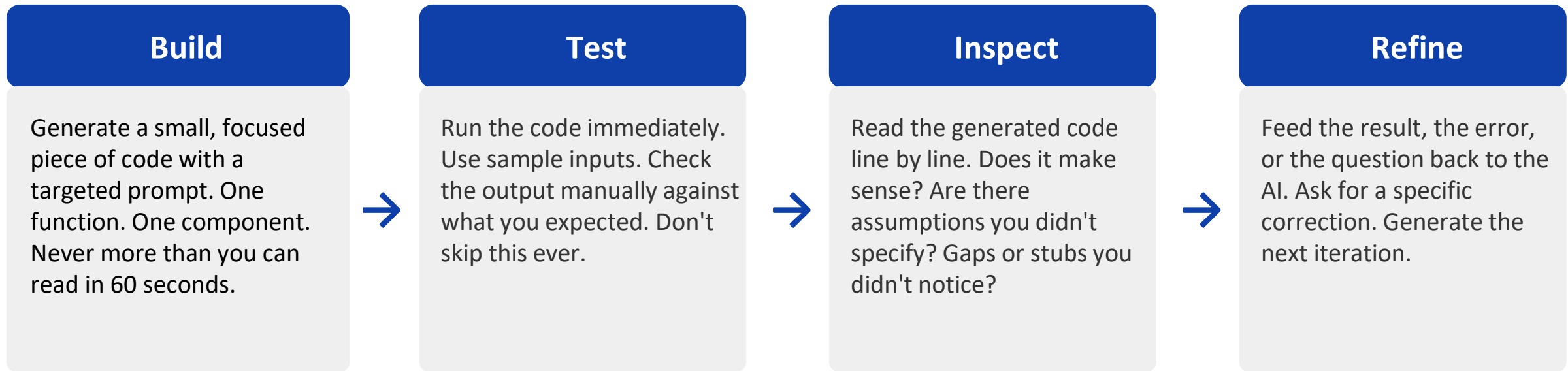
Why This Step Matters

- You can correct the plan before a single line of code is written.
- You can add missing components (e.g., "also include NI contributions").
- You set the context for all subsequent prompts in this session.
- The AI now has a shared map of the project—fewer surprises later.

The Iterative Development Loop

Build → Test → Inspect → Refine and repeat

Four-Step Iteration Loop



After Refine, go back to Build. The loop repeats until the component passes all your tests and makes sense when you read it.

Why Short Iterations Beat Long Ones—Every Time

Long Iteration (One Big Pass)

- Generate 200 lines of code in one go
- One bug hidden deep inside five functions
- Hours to find the root cause
- Hard to tell which part of the prompt caused the error
- Hard to re-prompt without rewriting everything

Short Iterations (Vibe Coding Style)

- Generate 15-20 lines for one function
- Bug appears immediately in the code you just read
- Minutes to identify and fix
- Easy to trace: this prompt → this error
- One targeted re-prompt fixes the specific issue

Short iterations keep errors small and contained. This rhythm is what allows you to use AI speed without sacrificing control.

The Prompt Templates Library

Four reusable patterns that cover 90% of what
you'll need

Prompt Templates 1 and 2: Planning and Task Breakdown

Template 1: Planning Prompt

"I want to build [PROJECT]. Before writing any code, give me a high-level plan — what components do I need and what should I build first?"

- Use at the very start of every new project.
- Gets AI thinking in components, not in code.
- Gives you a shared architecture to reference throughout.

Template 2: Task Breakdown Prompt

"Here is component [NAME] from our plan. Break it into individual implementation tasks. Keep each task small enough to implement in a single prompt."

- Run once per component, after the plan is confirmed.
- Each task becomes a prompt in the implementation phase.
- Small tasks = small errors = fast debugging

Use Templates 1 and 2 before writing a single line of code. They are your project architecture phase.

Prompt Templates 3 and 4: Implementation and Debugging

Template 3: Implementation Prompt

"Implement Task [N]: [DESCRIPTION]. Use Python. Keep the code simple and add comments explaining each section. Do not implement any other tasks yet."

- One task per prompt, never bundle tasks.
- "Do not implement any other tasks yet" prevents scope creep.
- Comments help you read and verify the output.

Template 4: Debugging Prompt

"Here is the code and the error it produces: [PASTE CODE] / [PASTE ERROR]. Explain what is wrong and provide a corrected version with comments."

- Always paste both the code AND the full error message.
- "Explain what is wrong" builds your understanding. Don't skip it.
- Then re-run and verify before moving on.

Save these 4 templates. They cover: plan → requirements → implementation → debugging—the full development cycle.

Live Demo: Implementing Task 1 of the Tax Calculator

Template 3 in Action: "Implement Task 1: Create a Python function `calculate_tax(income)` using 2024/25 UK tax brackets. Add comments. Do not implement anything else yet."

What Good AI Output Looks Like

- Function is named exactly as specified:
`calculate_tax(income)`
- Brackets are correct and clearly commented.
- Returns a value, not just prints it.
- No other functions added. Scope was respected.
- Comments explain each section clearly.

Your Review Checklist

- Does it run without errors?
- Does it return the right value for £20,000?
- What about £0? What about -£100?
- Any assumptions not in your prompt?

When AI Starts to Drift, How to Bring It Back

Signs That AI Is Drifting

- It builds beyond the task you asked for (scope creep).
- It contradicts decisions made earlier in the conversation.
- It forgets constraints or specifications from earlier prompts.

Technique 1: Explicit Boundaries

End every implementation prompt with: "Do not implement anything beyond this task. Do not modify any existing code."

Technique 2: Reset and Re-anchor

Start a new chat session. Paste your PROJECT_SPEC.md or a summary of decisions made so far. Re-establish shared context.

Technique 3: Summarise and Confirm

Ask: "Before we continue, summarise what we have built so far and what our next task is." This forces the AI to re-read the context.

Drift is not a failure—it is a signal. Short iterations catch it early. Long sessions let it compound.

Key Takeaways

01

Structure Beats Cleverness

Prompting is not about phrasing tricks. It is about staged workflows: plan → requirements → tasks → implement one at a time.

02

Short Loops Win

The iterative loop: Build, Test, Inspect, Refine keeps errors small and keeps you in control of the outcome.

03

Templates Save Time

Four proven templates cover planning, breakdown, implementation, and debugging. Use them on every project, every time.

Next up: Video 3—Testing and Evaluating: How to debug with AI and maintain human oversight

Summary

Here's a brief recap:

- Prompting as a structured skill defines how controlled inputs improve reliability over monolithic prompts.
- The staged workflow organizes development into sequential steps from planning to implementation.
- The iterative loop establishes a repeatable cycle for building and validating code.
- Prompt templates standardize how tasks are planned, executed, and debugged.
- The tax calculator example represents a structured application of these concepts in practice.

Learning Outcomes

You should now be able to:

- Decompose a problem into plan, requirements, tasks, and implementation stages.
- Execute the Build → Test → Inspect → Refine loop to iteratively develop code.
- Construct structured prompts using planning, implementation, and debugging templates.
- Diagnose and correct issues such as scope creep, errors, and context drift in AI outputs.
- Develop a simple application workflow (e.g., tax calculator) using staged prompting.